



OASIS Security Report

JWT Implementation Flaws Security Analysis

Date: 2026-06-12 00:38:27 — **Model:** llama3.2:3b

Vulnerability: JWT Implementation Flaws

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **9** file(s) for JWT Implementation Flaws.

[Return to table of contents](#)

Statistics

Files analyzed: 9 **Total findings:** 13

Critical



3

High		1
Medium		7
Low		2

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol</code>	0.672
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>	0.629
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.626
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.623
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>	0.619
<code>/work/project/dataset/vulnerable/reentrancy/simple_dao.sol</code>	0.614
<code>/work/project/dataset/vulnerable/access-control/mycontract.sol</code>	0.613
<code>/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol</code>	0.597
<code>/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol</code>	0.589

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.672

Source lines: 1-70

► Finding 1.1: Insecure JWT Algorithm (Medium)

Explanation

The contract uses a custom algorithm for quote, which is not secure.

Impact

Unauthorized access to token claims

Entry point

/_getOracleQuote

HTTP methods

GET

Exploitation steps

- Get oracle quote using custom algorithm

Remediation

Use secure algorithm like HS256 or RS256

File 2: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.629

Source lines: 1-225

► Finding 2.1: Lack of Signature Verification (Medium)

Vulnerable code

```
function revoke(bytes32 _operation) external {
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];
    // make sure they're an owner
    if (ownerIndex == 0) return;
    uint ownerIndexBit = 2**ownerIndex;
    var pending = m_pending[_operation];
    if (pending.ownersDone & ownerIndexBit > 0) {
        pending.yetNeeded++;
        pending.ownersDone -= ownerIndexBit;
        Revoke(msg.sender, _operation);
    }
}
```

Explanation

The `revoke` function does not verify the signature of the message sender.

Impact

Authentication bypass

Entry point

/WalletAbi/revoke(bytes32)

Parameters

msg.sender

Exploitation steps

- Send a revoked bytes32 value with a random signature

Conditions

msg.sender is an owner

Remediation

Implement signature verification using `require(msg.sender)` or similar

Secure example

```
function revoke(bytes32 _operation) external requires(msg.sender) {
    // ... }
```

Source lines: 126–467

► Finding 2.1: Weak JWT Secret (Low)

Vulnerable code

```
"_walletLibrary = 0xcafecafecafecafecafecafecafecafe;"
```

Explanation

Using a hardcoded secret in plaintext makes it easily accessible to attackers.

Impact

Unauthorized access to the wallet

Entry point

```
/initWallet(address[],uint256,uint256)
```

Execution path

```
Delegatecall(msg.data, 0x0, add(argsize, 0x4), 0x0, 0x0) ->
_walletLibrary.delegatecall(msg.data);
```

Parameters

_walletLibrary

Exploitation steps

- Obtain the value of `\_walletLibrary` through any means.

HTTP raw requests (Burp / ZAP)

```
GET /initWallet(address[],uint256,uint256) HTTP/1.1
```

```
Host: example.com
```

```
Accept: application/json
```

Source lines: 207–477

- eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWI0IjI0IjE0IiwiaWF0IjE0IiwibmFtZSI6IkpvaGFnIjoieXNDQyNzQ0NCIsImV4cCI6MTYxNzU0Tg50X0.eyJxL0RnB1KgHw8sT8Dk7w==

HTTP raw requests (Burp / ZAP)

POST /execute HTTP/1.1

Host: example.com

Content-Type: application/json

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaGFuIjoibmFtZSI6IkpvaGFuIiwiaWF0IjE5ODQyNzg4NCIsImV4cCI6MTYxNzU2OTg5ODk7w==

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.626

Source lines: 1-39

► Finding 3.1: JWT Algorithm Confusion Attack (Medium)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw)())
```

Explanation

The function uses `msg.sender.call.value()` which can be confused with the `msg.sender` in a Reentrancy attack.

Impact

Potential privilege escalation or session hijacking.

Entry point

withdrawFunds (uint256 _weiToWithdraw) public

Parameters

_weiToWithdraw

Exploitation steps

- Attacker sends a call to `msg.sender` with a malicious function pointer.

Conditions

Authenticated user

Remediation

Use the correct signature verification and algorithm selection.

Secure example

```
require(msg.sender.call.value(_weiToWithdraw, _signature))
```

File 4: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.623

Source lines: 1-85

► Finding 4.1: Insecure JWT signing (Medium)

Explanation

The contract uses the SHA-256 algorithm without proper key management, making it vulnerable to signature confusion attacks.

Impact

Potential for unauthorized access

Entry point

/CashOut(uint _am)

Execution path

```
```  
1. Call "/CashOut(uint _am)"
2. Verify signature
3. Update balance
```
```

Parameters

_am

Exploitation steps

- Get the contract address

HTTP raw requests (Burp / ZAP)

```
POST /CashOut(uint) HTTP/1.1  
Host: <contract_address>  
Content-Type: application/json  
  
{  
  "_am": 10  
}
```

File 5: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.619

Source lines: 1-227

► Finding 5.1: Insecure JWT signing (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external returns
(bytes32 o_hash);
function confirm(bytes32 _h) returns (bool o_success);
```

Explanation

The contract uses a simple SHA-256 function for JWT signing without verification.

Impact

Attacker could manipulate the JWT payload to gain unauthorized access.

Entry point

/execute

Execution path

```
execute -> confirm -> o_hash
confirm -> o_success
```

Parameters

_data

Exploitation steps

- Obtain a valid JWT payload

Example payloads

- `{'op': 'transfer'}`

HTTP raw requests (Burp / ZAP)

```
POST /execute HTTP/1.1
Host: <address>
Content-Length: 0
```

Source lines: 1-286

► Finding 5.1: Vulnerable Signature Verification (Critical)

Vulnerable code

```
function revoke(bytes32 _operation) external { ... }
```

Explanation

The revoke function does not verify the operation signature, allowing an attacker to manipulate the operation hash.

Impact

An attacker can bypass authentication and execute arbitrary operations.

Entry point

/revoke

Execution path

```
function revoke(bytes32 _operation) { if (_operation != sha3(_msg.data))  
throw; }
```

Parameters

_operation

Exploitation steps

- Manipulate the operation hash to bypass verification

Example payloads

- `sha3(' invalid ')`

Conditions

Contract owner has access and is authenticated.

Remediation

Implement signature verification for all operations.

Secure example

```
function revoke(bytes32 _operation) external { require(_operation ==  
sha3(msg.data)); }
```

Source lines: 96-414

► Finding 5.1: Lack of JWT signature verification (Medium)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
  if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0)
  {
```

Explanation

This function does not verify the JWT signature before confirming the transaction.

Impact

A malicious attacker could spoof a valid JWT and gain unauthorized access to the contract.

Entry point

/confirm

Execution path

```
confirm() {
  // ... (no signature verification)
}
```

Parameters

m_txs[_h].to, m_txs[_h].value, m_txs[_h].data

Exploitation steps

- Calculate a new JWT with the same payload and header but a different signature
- Send the new JWT to the contract

Example payloads

- new JWT with spoofed signature

Conditions

Contract is not properly initialized or configured.

Remediation

Verify the JWT signature before confirming the transaction using a secure library.

Secure example

```
function confirm(bytes32 _h) onlymanyowners(_h) returns (bool o_success) {
  require(verify(_h));
}
```

File 6: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.614

Source lines: 1-35

► Finding 6.1: Lack of Signature Verification (Critical)

Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

Explanation

The contract does not verify the JWT signature, making it vulnerable to algorithm confusion attacks.

Impact

Authentication bypass and privilege escalation

Entry point

/withdraw(uint)

HTTP methods

POST

Exploitation steps

- Reconstructing the JWT header and payload to forge a valid signature

Secure example

```
bool res = verifyJWT(msg.sender.call.value(amount)()) || msg.sender == owner;
```

File 7: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.613

Source lines: 1-31

► Finding 7.1: Lack of JWT signature verification (Medium)

Vulnerable code

```
require(tx.origin == owner);
```

Explanation

The contract does not verify the JWT signature, making it vulnerable to algorithm confusion attacks.

Impact

Session hijacking or unauthorized access

Entry point

/sendTo

Execution path

```
tx.origin == owner; receiver.transfer(amount);
```

HTTP methods

POST

Parameters

amount

Exploitation steps

- Send a malicious JWT with a different signature
- Use an intermediary contract to swap the ownership

Conditions

tx.origin == owner

Remediation

Verify the JWT signature using a secure library or custom implementation.

Secure example

```
require(tx.origin.verify());
```

File 8: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.597

Source lines: 1-61

► Finding 8.1: Weak JWT signing key (Low)

Explanation

The contract uses a fixed string as its secret, making it vulnerable to brute-force attacks.

Impact

Unauthorized access

Entry point

/play(uint number)

Execution path

```
getProfit() -> andTheWinnerIs() -> delete players; play() -> play()
```

Parameters

number, msg.value

Exploitation steps

- Get the current secret value from memory

Example payloads

- 

Conditions

Contract has a weak signing key

Remediation

Use a secure random number generator to generate keys.

File 9: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.589

Source lines: 1-42

► Finding 9.1: Vulnerability found (Critical)

Explanation

Missing signature verification for JWT token, allowing potential attacks.

Impact

Authentication bypass or unauthorized access

Entry point

/setReward() public payable

Execution path

```
sig1 -> checkSig -> if ok -> transfer owner msg.value; else return invalid
```

HTTP methods

POST

Exploitation steps

- Use a tool to inject a malicious signature

Conditions

Authenticated user can reach /setReward() with a valid token

Remediation

Implement proper JWT signature verification using a secure library or function.

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)